

torch.nanmean#

<https://pytorch.org/docs/stable/generated/torch.nanmean.html>

Description:

Computes the mean of all non-NaN elements along the specified dimensions. Input must be floating point or complex. This function is identical to `torch.mean()` when there are no NaN values in the input tensor. In the presence of NaN, `torch.mean()` will propagate the NaN to the output whereas `torch.nanmean()` will ignore the NaN values (`torch.nanmean(a)` is equivalent to `torch.mean(a[~a.isnan()])`). If `keepdim` is `True`, the output tensor is of the same size as input except in the dimension(s) `dim` where it is of size 1. Otherwise, `dim` is squeezed (see `torch.squeeze()`), resulting in the output tensor having 1 (or `len(dim)`) fewer dimension(s).
input (Tensor) – the input tensor, either of floating point or complex dtype
dim (int or tuple of ints, optional) – the dimension or dimensions to reduce. If `None`, all dimensions are reduced.

Function Signature

```
torch.nanmean(input, dim=None, keepdim=False, *, dtype=None, out=None) → Tensor#
```

Parameters

Keyword Arguments

Parameters

Keyword

`dtype` (`torch.dtype`, optional) – the desired data type of returned tensor. If specified, the input tensor is casted to `dtype` before the operation is performed. This is useful for preventing data type overflows. Default: `None`. `out` (`Tensor`, optional) – the output tensor.

Code Examples

```
>>> x = torch.tensor([[torch.nan, 1, 2], [1, 2, 3]])
>>> x.mean()
tensor(nan)
>>> x.nanmean()
tensor(1.8000)
>>> x.mean(dim=0)
tensor([ nan, 1.5000, 2.5000])
>>> x.nanmean(dim=0)
tensor([1.0000, 1.5000, 2.5000])

# If all elements in the reduced dimensions are NaN then the
# result is NaN
>>> torch.tensor([torch.nan]).nanmean()
tensor(nan)
```

Notes & Warnings

NOTE

See also `torch.mean()` computes the mean value, propagating NaN.

Detailed Documentation

Documentation

Computes the mean of all non-NaN elements along the specified dimensions. Input

must be floating point or complex.

This function is identical to `torch.mean()` when there are no NaN values in the input tensor. In the presence of NaN, `torch.mean()` will propagate the NaN to the output whereas `torch.nanmean()` will ignore the NaN values (`torch.nanmean(a)` is equivalent to `torch.mean(a[~a.isnan()])`).

If `keepdim` is `True`, the output tensor is of the same size as input except in the dimension(s) `dim` where it is of size 1. Otherwise, `dim` is squeezed (see `torch.squeeze()`), resulting in the output tensor having 1 (or `len(dim)`) fewer dimension(s).

`input (Tensor)` – the input tensor, either of floating point or complex dtype

`dim (int or tuple of ints, optional)` – the dimension or dimensions to reduce. If `None`, all dimensions are reduced.

`keepdim (bool, optional)` – whether the output tensor has `dim` retained or not. Default: `False`.

`dtype (torch.dtype, optional)` – the desired data type of returned tensor. If specified, the input tensor is casted to `dtype` before the operation is performed. This is useful for preventing data type overflows. Default: `None`.

`out (Tensor, optional)` – the output tensor.

`torch.mean()` computes the mean value, propagating NaN.